

Das kleine 1×1 barrierefreier Websites: UI-Design

Eine Handreichung von Annika Brinkmann. Stand: 26. Mai 2025

Diese Handreichung zeigt die Möglichkeiten von Designer:innen zur Verbesserung der Barrierefreiheit auf und stellt die notwendigen Handlungsoptionen und Informationen vor.

Da es viele Überschneidungen mit anderen Rollen und Fachbereichen gibt, liegt dieser Handlungsempfehlung ein **Glossar** bei, das die Abkürzungen und Fachbegriffe erklärt.

Links speziell zu den hier behandelten Themen befinden sich jeweils am Ende eines Abschnittes. Weitere Links im Dokument [Handout zum Thema Barrierefreiheit](#).

Wissenswertes zu CSS befindet sich im [1x1 CSS Cheat-Sheet](#)

Inhaltsverzeichnis dieser Handreichung

Präambel: Einleitung zur digitalen Barrierefreiheit	3
Abgrenzung: Einflussbereich von Designer:innen	3
“Goldene Regeln” im (barrierefreien Web-) Design	4
Gutes Design ist penibel aufgeräumt	4
Gutes Design ist optimal lesbar	5
Gutes Design setzt Farben gezielt ein	7
Gutes Design nutzt korrekte Satz- und Sonderzeichen	9
Wichtige Kennzahlen im barrierefreien Webdesign	11
Wichtige Größenangaben	11
Vorgaben für gute, barrierefreie Lesbarkeit	12
Sinnvolle Abstände angeben	14
Farbkontraste	16
Das “Favicon” Favoriten-Icon	17
Implementierung	17
Aspekte der Barrierefreiheit	17
Beispiele	17
Überschneidungen mit Apps	17
Links und Materialien	17
Richtungen anzeigen und nutzen	18
Implementierung	18
Aspekte der Barrierefreiheit	18
Überschneidungen mit Apps	18
Beispiele	18
Links und Materialien	18
UI-Element Hero-Image (ohne Text)	19

Implementierung	19
Aspekte der Barrierefreiheit	21
Überschneidungen mit Apps	21
Beispiele	21
Links und Materialien	21
UI-Element Hero-Image (mit Text)	22
Implementierung	22
Aspekte der Barrierefreiheit	23
Überschneidungen mit Apps	23
Beispiele	23
Links und Materialien	24
UI-Elemente Karussells	25
Implementierung	25
Aspekte der Barrierefreiheit	26
Überschneidungen mit Apps	26
Beispiele	26
Links und Materialien	26
UI-Element Sticky Elements	27
Implementierung	27
Aspekte der Barrierefreiheit	29
Überschneidungen mit Apps	29
Beispiele	30
Links und Materialien	30
Inhalts-Elemente von Webseiten	31
Äußere Strukturelemente	31
Struktur-Elemente – Landmarks	32
Block-Elemente für einfache Inhalte	35
Block-Elemente für komplexe Inhalte mit Unter-Tags	35
Inline-Elemente	36
Anhang	37
Glossar	37
Hilfreiche Online-Tools und Quellen für digitales Design	38
Input – was fehlt noch	40

Präambel: Einleitung zur digitalen Barrierefreiheit

Um Barrierefreiheit wenigstens rudimentär messen zu können, wurden bereits 1999 vom W3C-Consortium die WCAG¹-Richtlinien veröffentlicht. Auf diesen basieren die Regelungen der EU (EN 301 549) und die wiederum bilden die Grundlage für die BIK BITV² Prüfschritte.

In diesem Dokument werden teilweise Prüfschritte bzw. Richtlinien behandelt. Da noch nicht alle WCAG-Richtlinien aus der Version 2.2 (Herbst 2023) in die BITV 2.0 Einzug erhalten haben, sind sie teilweise im englischen Original verlinkt.

Nicht alle Punkte basieren auf Prüfschritten, einige davon beziehen sich auch auf Best-Practice-Beispielen und eigene Beobachtungen.

Abgrenzung: Einflussbereich von Designer:innen

Webseiten zu erstellen ist ein Gemeinschaftsprojekt von Menschen mit verschiedenen Rollen, Kenntnissen und Fertigkeiten.

Phase	Rolle	Tätigkeiten/Verantwortlichkeit
Research & Konzeption	UX-Design	Recherche, Konzept, (Usability-)Tests, Wireframes
	UI-Design	Farben, Schriften, Icons, Visuals festlegen
Entwicklung & Testing	Frontend	HTML, CSS, JavaScript
	Backend	CMS, Funktionen
Betrieb	Redaktion	Inhalte erstellen, Inhalte im CMS einfügen
	Bildredaktion	Visuals erstellen, Bilder auswählen, Alt-Texte

Da digitale Barrierefreiheit alle Bereiche betrifft, können die einzelnen Rollen nur einen eingeschränkten Einfluss auf die Qualität der Barrierefreiheit insgesamt ausüben.

Auch wenn der Einfluss der Einzelner begrenzt ist und nicht alle Entwicklungen parallel laufen, so ist jede Maßnahme zur Verbesserung der Barrierefreiheit wichtig!

UI-Designer:innen haben generell einen großen Einfluss auf:

- Die Farben und Farbkontraste (in Abstimmung mit CD-Manual)
- Die Darstellung in verschiedenen Anzeige-Modi => siehe RIAN
- Schrift-Wahl, Schriftgrößen (bedingt) und (responsives) Layout
- Einsatz von Bildern, Icons, Illustrationen und Animationen

In einigen Details müssen sich UI-Designer:innen jedoch mit Vorgaben arrangieren, z.B.:

- Mindestgröße von Linkflächen sind vorgegeben
- Verwendung von Grafiken für Links und Buttons weglassen bzw. Text ergänzen
- Darstellung von Labels in Formularen **über** den Eingabefeldern angezeigt werden

¹ WCAG-Kriterien (Englisch) <https://www.w3.org/WAI/WCAG22/quickref>

² BIK BITV Prüfschritte <https://bitvtest.de/pruefverfahren/bitv-20-web>

“Goldene Regeln” im (barrierefreien Web-) Design

Gutes Design:

1. [ist penibel aufgeräumt](#)
2. [ist optimal lesbar](#)
3. [setzt Farben gezielt ein](#)
4. [nutzt korrekte Satz- und Sonderzeichen](#)

Gutes Design ist penibel aufgeräumt

Satzspiegel: Abstand des Textbereiches zum Rand

Die Abstände von Bildern und Textblöcken zum Rand können je nach Skalierung der Schrift, Zoomfaktor und Bildschirmbreite unterschiedlich sein, jedoch innerhalb der gleichen Darstellungsgröße sind sie gleich.

Nur in Ausnahmen sind Abweichungen von der vertikalen Linie sinnvoll, z.B. bei:

- a. Hero-Images
- b. Zitaten
- c. Aufzählungszeichen
- d. Navigationselementen im Header (nur wenn unbedingt erforderlich)

Abstände innerhalb des Satzspiegels

Auch innerhalb des Satzspiegels sollten Abstände in der Vertikalen und Horizontalen konsistent sein. Z.B. Texte innerhalb einer Box (mit oder ohne Bild): Innenabstand 16px zu allen Seiten!

Textsatz

wird an der Seite ausgerichtet, wo mit Lesen begonnen wird:

- a. Links in Sprachen, die von links nach rechts gelesen werden
- b. Rechts in Sprachen, die von rechts nach links gelesen werden

Gutes Design ist optimal lesbar

Es gibt verschiedene Anforderungen an Schriften und verschiedene Lehren, welche Schrift für wen und unter welchen Bedingungen welche Schrift besser lesbar ist.

Serifen, Serifenlos, Monospace oder Fantasy?

Im Print wird häufig eine Schrift mit Serifen empfohlen, die die Buchstaben besser miteinander verbinden sollen. Diesen Schriften wird nachgesagt, dass sie nicht für Bildschirme geeignet sind. Dies stimmt nicht mehr unbedingt, seitdem Bildschirme eine höhere Auflösung haben als im Druck nötig ist.

Schriftwahl allgemein

- a. Schriften mit offenen, gut unterscheidbaren Buchstaben: e c besser als e c
- b. Schriften mit oder ohne Serifen:
mit Serifen: Ii
ohne Serifen: li
- c. Handschriften nur in Druckbuchstaben, ggf. Buchstabenabstand³ erhöhen:
Gut: **Hallo**
Weniger gut: *Hallo*
- d. Schriften mit Tabellenziffern⁴ (gleiche Breite für alle Ziffern):
1.111,11 Besser als: 0.111,11
0.000,00 0.000,00

Schriftwahl für leichte Sprache

- a. Schriften mit extra guter Differenzierung von db und qp, li, o0O
- b. Schriften mit Majuskelziffern⁵ (alle Zahlen zwischen Grundlinie und Oberlänge)

Schriftschnitte

- a. Normal und **Bold** visuell gut unterscheidbar
- b. **Nicht** einsetzen:
 - i. Dünne Schriftschnitte (Light oder Thin)
 - ii. schmal laufende (Condensed)

Schriftauszeichnungen

- c. Einsetzen: im Code, **bold** visuell darstellen – wenn vorhanden⁶!
- d. **Nicht** einsetzen:
 - i. *kursiv* bzw. *italic* Schrift
(hier falsch dargestellt, am “a” erkennbar, das sollte so aussehen: **a**)

Ligaturen s+z = ß bzw. fi oder ffi zusammengezogen sind typographisch korrekt, können aber für Menschen mit geringen Lesekompetenzen schwer verständlich sein und sollten daher nicht in der leichten Sprache eingesetzt werden.

Weitere Angaben zu Schriftgröße, Zeilenabstand sind im Kapitel “[Vorgaben für gute, barrierefreie Lesbarkeit](#)” zusammengestellt!

³ <https://developer.mozilla.org/en-US/docs/Web/CSS/letter-spacing>

⁴ <https://www.typolexikon.de/tabellenziffern/>

⁵ <https://www.typolexikon.de/majuskelziffern/> vs. <https://www.typolexikon.de/minuskelziffern/>

⁶ <https://alistapart.com/article/say-no-to-faux-bold/> +
<https://www.smashingmagazine.com/2012/07/avoiding-faux-weights-styles-google-web-fonts/>

Gutes Design setzt Farben gezielt ein

Es gibt viele Faktoren, die die Farbwahl beeinflussen. Hier ein Überblick:

1. **Corporate Colors – Brandfarben** “können” teilweise nicht einfach geändert werden
2. **Psychologische Farbwirkung**⁷
3. **Farbwahrnehmung** (und Bedeutung) ist auch kulturell unterschiedlich
4. **Farbsehschwächen** müssen berücksichtigt werden
5. **Farbwiedergabe** auf den unterschiedlichen Bildschirmen/Medien sehr verschieden!
Generell Unterschied zwischen CMYK (Offset-Druck) und RGB (Bildschirme)
6. **Kopierwerkzeuge und Darstellungstechniken**
 - a. Gelb ist zu hell um gefaxt zu werden (und hat auf weiß zu wenig Kontrast für die Barrierefreiheit)
 - b. E-Book-Reader haben oft einen S/W-Bildschirm
7. **Gewünschte Kontrastart**⁸
 - a. Komplementärkontrast
 - b. Farbe-an-sich-Kontrast
 - c. Hell-Dunkel-Kontrast
 - d. Kalt-Warm-Kontrast
 - e. Quantitätskontrast
 - f. Qualitätskontrast
 - g. Simultankontrast
 - h. Bunt-Unbunt-Kontrast
8. **RIAN-Kriterien**
 - a. Farben für Light- und Darkmode
 - b. höherer und niedrigerer Kontrast
Davon unabhängig: Forced Colors-Modus
9. **Vorlieben von Stakeholdern**
"Mein Spouse mag die Farbe A lieber als B!"
10. **Branchenübliche Farben**
 - a. Beratung Blau (weil seriös)
 - b. Naturkosmetik Grün (naturverbunden) und Pastelltöne (wenig invasiv)
11. **Markenrecht**
 - a. geschützte Logos haben oft feste Farben mit hinterlegt!
 - b. Online- und Telekommunikationsunternehmen: kein Magenta nutzen!
(wegen Patent der Telekom)
12. **Signalfarben**
was, wenn die negative Signalfarbe (z.B. Rot) der Markenfarbe (z.B. bei Bosch) entspricht?
13. **Grauwert und Helligkeit**⁹
14. **Kontraste der Farben zueinander**
15. **Energie-Effizienz der Farben**¹⁰

⁷ z.B. auf <https://farbtonkarte.de/>

⁸ gut zusammengefasst auf <https://www.webdesign-journal.de/farbkontraste/>

⁹ Zu Grauwerten und Helligkeiten von Farben: <https://iamkate.com/data/12-bit-rainbow/>

¹⁰ <https://greentheweb.com/ideen-fur-energieeffiziente-farbpaletten/>

Welche Farben braucht es für eine Webseite

Diese Sammlung ist nicht vollständig und ggf. sind manche Farben auch identisch mit Farben von anderen Bereichen/Einsatzgebieten.

Alles für Light- und Dark Mode!

Hintergrund-Farbe (Einfarbig oder z.B. mit Farbverlauf) für

- body
- header
- main
- footer
- aside
- hervorgehobene Bereiche
- + ggf. Border-Colors

Textfarbe

- Fließtext (Paragraphen)
- Logo + Links im header
- Text und Links im footer
- Überschriften
- Links – :any-link, :hover, :visited, :focus (outline)
- Unterstreichungen

Textfarbe mit extra definierter Hintergrundfarbe

- Selektierter Text
- markierter Text
- hervorgehobene Bereiche

Icons + Piktogramme

- Einfarbig oder bunt? Immer genug Kontrast zum Hintergrund und untereinander!

Formulare

- Hintergrundfarbe + Textfarbe von Inputfeldern
- Caret-Color (Cursor im Eingabefeld)
- ggf. :empty, :user-valid, :user-invalid

Status-Farben (immer + Form!)

- positive Mitteilungen
- negative Mitteilungen
- neutrale Mitteilungen

Buttons

- primäre Buttons
- sekundäre Buttons
- tertiäre Buttons

Schmuckfarben

- z.B. "umgeknickte Ecke" bei bestimmten Textfeldern

Gutes Design nutzt korrekte Satz- und Sonderzeichen

Nicht alle korrekten Satzzeichen sind direkt auf jeder Tastatur gut erreichbar/vorhanden. Im folgenden Kapitel können die korrekten Zeichen kopiert werden. Mit UTF-8-Codierung sind sie oft direkt im Text einsetzbar. Um ganz sicherzugehen, dass Webseiten die Zeichen auch in anderen Kodierungen korrekt darstellen können die HTML-Entities¹¹ verwendet werden, die bei Ascii-Kodierung auch für Umlaute und Ligaturen benötigt werden. Ob die HTML-Schreibweise `…`; oder die "sprechende" Version `&hellip`; für den Dreipunkt genutzt werden, ist Geschmackssache. Wenn – in seltenen Fällen – der Einsatz als Generated Content im CSS sinnvoll ist, muss es jedoch die CSS-Schreibweise `\2026` sein!

Gedankenstrich vs. Divis und Minuszeichen

Die **Divis** – auch Spiegel- oder Bindestrich¹² – ist ein kurzer Strich, in diesem Satz hinter Spiegel zu sehen. Der lange Strich, der im Satz vor diesem hier den Einschub startet und beendet, ist ein **Gedankenstrich**¹³.

Die **Divis** "–" kommt zum Einsatz:

- Wenn es sich um eine Aufzählung handelt – allerdings ist dafür im HTML das Listenelement vorgesehen, weshalb auch dieses Beispiel eine Liste ist!
- Wenn eine Wort-Kette gebildet wird
- Wenn ein Wort- oder Zeichenketten-Teil gemeint ist
- Zwischen Doppel-Namen, auch bei Firmen-Bezeichnungen

Der **Gedankenstrich** "–" HTML `&ndash`; kommt zum Einsatz:

- Wenn Leerzeichen davor und danach kommen – z.B. hier!
- Bei Zahlen und Uhrzeiten wird er für Bereiche N1–N2 oder 13:00–15:00 Uhr eingesetzt. Letzteres ist auf Öffnungszeiten-Schildern sehr oft falsch gesetzt.

Im **Englischen** wird der Geviertstrich¹⁴ ohne Leerzeichen als Gedankenstrich verwendet:

- the fire drill—it was chaos¹⁵

Auslassungszeichen Dreipunkt

Wenn Wörter oder Textpassagen ausgelassen werden, ist das korrekte Zeichen der Dreipunkt ...¹⁶

¹¹ HTML-Entities <https://symbbl.cc/de/html-entities/>

¹² Bindestrich: <https://symbbl.cc/de/002D/>

¹³ Gedankenstrich: <https://symbbl.cc/de/2013/>

¹⁴ Geviertstrich: <https://symbbl.cc/de/2014/>

¹⁵ https://de.wikipedia.org/wiki/Geviertstrich#Gedankenstrich_im_Englischen

¹⁶ Dreipunkt: <https://symbbl.cc/de/2026/>

Geschütztes Leerzeichen

Manchmal werden Leerzeichen benötigt, die zu keinem Zeilenumbruch führen. Vor allem im responsiven Webdesign ist dies relevant, da Zeilen sehr unterschiedlich lang ausfallen können. Je nach Schriftgröße manchmal nur 6 Zeichen, manchmal über 75!

Das geschützte Leerzeichen¹⁷ kann z.B. gesetzt werden:

- **vor** dem Gedankenstrich, damit es nicht als erstes Zeichen in der neuen Zeile steht:
So ein Leerzeichen kann – sofern es sinnvoll eingesetzt wird – viel bewirken!
- zwischen den (kurzen!) Teilen eines Markennamens, damit er nicht getrennt wird:
MarkenName

Mehrere geschützte Leerzeichen zur Steuerung von Einrückungen sind nicht zulässig!

Adresse: Musterstraße 13

Ort: Springfield

Geschützter Trennstrich

Bei Firmen-Namen mit Bindestrich kann es ebenfalls sinnvoll sein, einen Zeilenumbruch zu verhindern.

HTML: Firmen-Name

Beispiel: Firmen-Name

HTML-Entity: -

Unicode: U+2011¹⁸

Mittelpunkt

Der Mittelpunkt wird in der Leichten Sprache genutzt, um das Zusammensetzen von Hauptwörtern visuell zu verdeutlichen. Hier ist es wichtig, NICHT den Bulletpoint • zu verwenden. Korrekt sind der Mediopunkt¹⁹ · oder der Wortseparator mittlerer Punkt²⁰. Damit wird aus dem Blumentopferde die besser lesbare Blumentopf·erde.

Mathematische Zeichen

Wenn es um ein Minuszeichen²¹ geht, sollte nicht die Devis genutzt werden, sondern auch hier das korrekte Minus “-”.

4-2 = 2 HTML −

Divisionszeichen²² “÷” 4÷2 = 2 HTML ÷

Multiplikationszeichen²³ “×” 4×2 = 8 HTML ×

Plus + und Gleichheitszeichen = sind auf Standard-Tastaturen zu finden.

¹⁷ Geschütztes Leerzeichen: <https://symbbl.cc/de/00A0/>

¹⁸ Geschützter Trennstrich: <https://symbbl.cc/de/2011/>

¹⁹ Mediopunkt: <https://symbbl.cc/de/00B7/>

²⁰ Wortseparator mittlerer Punkt: <https://symbbl.cc/de/2E31/>

²¹ Minuszeichen: <https://symbbl.cc/de/2212/>

²² Divisionszeichen: <https://symbbl.cc/de/00F7/>

²³ Multiplikationszeichen: <https://symbbl.cc/de/00D7/>

Weitere mathematische Zeichen sprengen den Umfang an dieser Stelle. Sie sind auf [Symbbl.cc²⁴](#) zusammengefasst.

Komplexer Satz von mathematischen Formeln benötigt jedoch LaTeX²⁵ und ist nicht einfach barrierefrei umzusetzen, zumal die Ergebnisse oft als Bild exportiert werden.

Mathematische Formeln sollten eher anhand der Lautschrift gesetzt und getestet werden:

$$f(x) = (x + 3) \times (x/3)$$

Wichtige Kennzahlen im barrierefreien Webdesign

Die Angaben dieser Seite bilden das Gerüst für gutes barrierefreies Webdesign!

Alle px-Angaben beziehen sich auf Viewport-Pixel, nicht auf die Auflösung der Bildschirme!
Bei Größenangaben von Flächen gilt immer Breite * Höhe!

Wichtige Größenangaben

Umrechnung PIXEL <> REM

rem bezieht sich auf eine relative Basis-Einheit. Wenn im CSS der Seite nichts oder für das Root-Element HTML die Angabe `html{font-size: 100%}` hinterlegt wurde, entspricht 1rem jeweils der Schriftgröße, die in den Browser eingestellt ist. Der Standard ist 16px.

Bei der Umrechnung von Schriftgrößen in rem gilt also: **16px = 1rem**

Das ist vor allem für die Umsetzung in HTML wichtig!

Umrechnung PIXEL <> PUNKT

16px entspricht einer Größe von 12pt. Der Faktor ist also 1pt = 1.333px.

Der Prüfschritt WCAG 1.4.3 (BITV/EN 9.1.4.3²⁶) setzt für große Schrift ein Kontrastverhältnis von 1:3 voraus: "Für Schriftgrößen unter 24 px (beziehungsweise 18,7 px bei fetter Schrift) prüfen, ob das Kontrastverhältnis bei 4,5:1 oder größer liegt." – 24px entspricht 18pt, teilweise wird auch diese Größe genannt.

ACHTUNG: da Schrift variabel sein sollte bitte NICHT unter 4,5:1 gehen!

Screen-Breiten für responsive Design

Das Design muss letztlich zwischen **160px** und über **1440px** Viewport-Width funktionieren. Damit es darauf ausgelegt wird bitte die folgenden Screenbreiten im Designprozess nutzen.

Tipp: Immer in 100% Zoom arbeiten, um ein Gefühl für sinnvolle Proportionen zu erlangen!

px	rem	Geräte-Art	CSS-Media-Query	Kriterium
320	20	Smartphone	default / @media (min-width: 20rem)	WCAG 1.4.10 AA

²⁴ Mathematische Zeichen: <https://symbbl.cc/de/collections/mathematical-signs/>

²⁵ <https://nats-www.informatik.uni-hamburg.de/pub/User/LaTeXNovember/Formel.pdf>

²⁶ <https://bitvtest.de/pruefschritt/bitv-20-web/bitv-20-web-9-1-4-3-kontraste-von-texten-ausreichend>

768	48	Tablet Hoch	@media (min-width: 48rem)	
1024	64	Tablet Quer	@media (min-width: 64rem)	
1280	80	Desktop klein	@media (min-width: 80rem)	BITV 9.1.4.10 AA
1440	90	Desktop groß	@media (min-width: 90rem)	

Hinweis: Bei Designs in 1440px Breite beachten, dass es hier vor allem darum geht festzulegen, wie breit Inhaltsbereiche, vor allem Zeilenlängen maximal werden dürfen!

Die Höhe der Screens bitte am Inhalt ausrichten! Bis einschließlich Footer sollte alles drauf sein!

Mindestgrößen von Klickflächen

Gelten auch für die Zeilenabstände von Listen, die Links enthalten. Ausschlaggebend ist die Klickfläche/der Abstand zum nächsten klickbaren Element, nicht die visuelle Darstellung. Share-Icons können z.B. 16*16 Pixel groß sein, die klickbare Fläche muss größer sein!

px	Bedienart	CSS-Media-Query	Kriterium
24*24	Mausbedienung	default oder @media (pointer: fine)	WCAG 2.5.8 AA
44*44	Finger-Bedienung	@media (any-pointer: coarse)	WCAG 2.5.5 AAA

Wichtig: Die Klickflächen müssen auch dann eingehalten werden, wenn Nutzende eine kleine Schriftgröße im Browser eingestellt haben!

Ebenfalls wichtig: Diese Größen im CSS in Pixeln und **NICHT** als 1.5rem bzw. 2.75rem angeben, da sie sonst mit der Größe der eingestellten Schrift skalieren.

Noch mal zur Veranschaulichung:

Schriftgröße	Mausnavigation	Touch Navigation	Anmerkung
9px	mindestens 24*24px	mindestens 44*44px	minimale Skalierung
16px	mindestens 24*24px	mindestens 44*44px	Standard-Schriftgröße
32px	mindestens 24*24px	mindestens 44*44px	200%-Skalierung
72px	mindestens 24*24px	mindestens 44*44px	Maximal einstellbar

Vorgaben für gute, barrierefreie Lesbarkeit

Schriftgrößen im Design

Das Design muss mit den aktuellen Tools in einer fixen Pixel-Größe erstellt werden. Es ist jedoch notwendig, dass diese Größe für die Umsetzung in rem umgewandelt wird. Nur dann passt sich die Schriftgröße an die Einstellungen der Nutzenden an.

px	rem	Text-Art	Kommentar, Best Practice-Code	Kriterium
= 16	1	Fließtext	inklusive dem "Kleingedruckten" auf allen Geräten: Mobil, Tablet, Desktop!	WCAG 1.1.4 AA
20–25	1.5	H1	clamp(1.25rem, 2.5vw, 1.5rem)	
19,2–24	1.4	H2	clamp(1.2rem, 2.4vw, 1.4rem)	
18.4–22.4	1.3	H3	clamp(1.15rem, 2.3vw, 1.3rem)	
17.6–20.8	1.2	H4	clamp(1.1rem, 2.2vw, 1.2rem)	
16.8–19.2	1.1	H5	clamp(1.05rem, 2vw, 1.1rem)	
16.8–19.2	1.1	H6	clamp(1.05rem, 2vw, 1.1rem)	
= 12	0.75	Sup/Sub	Hoch- und niedrig gestellte Texte + Ziffern ²⁷	Best Practice

Hinweis: Damit Menschen, die große oder kleine Schrift brauchen, sowohl Überschriften als auch Fließtext und Kleingedrucktes gut lesen können, ist es elementar wichtig, dass die Differenz der Größe nicht allzu riesig ausfällt!

Hinweis: für Leichte Sprache die gleichen rem-Werte verwenden, aber im Design 125% und im CSS `html[lang*="simple"]{font-size: 125%}` einsetzen!

Edge-Cases von Schriftgrößen

Die Schriftgrößen, die sich in den Browsern einstellen lassen, variieren. Basierend auf 16px bei 100% (entspricht Mittel in Chrome) sind die unten stehenden Werte die Maximalen:

px	Browser	Kriterium	Browser	px
4	Opera 25%	Für kleine Schrift gibt es kein Kriterium. WCAG 1.4.4 verlangt 200%, also 32px. Dies geht an den Bedürfnissen der Zielgruppe zum Teil vorbei.	Opera 500%	80
9	Chrome		Chrome	72

Es empfiehlt sich sowohl für 320px als auch für 1440px die Designs exemplarisch mit den Schriftgrößen 9px und 72px zu betrachten, damit Entwickelnde sehen worauf zu achten ist.

Wichtig für Umsetzung: auch bei großer Schrift soll keine horizontale Scrollbar entstehen!

Ein guter Zeilenabstand

- lässt Schrift visuell Luft zum atmen
- ermöglicht es, beim Lesen nicht in der Zeile zu verrutschen
- balanciert zwischen Corporate Design und Anpassungsfähigkeit

²⁷ z.B. für Fußnoten!

- unterschreitet NICHT den Wert von 1, da sonst speziell im forced-color-Mode mit der Hinterlegung die unteren Enden der Buchstaben abgeschnitten werden.

%	Faktor	Text-Art	Kommentar	Kriterium
150	1.5	Zeilenabstand Fließtext	Bei den meisten Schriften optimal!	1.4.12 Best Practice
125	1.3	H1 – H6	bzw. bei Handschriften größer!	Best Practice

Abstand zwischen zwei Paragraphen XXX

%	Faktor	Text-Art	Kommentar	Kriterium
150	1.5	Zeilenabstand Fließtext	Bei den meisten Schriften optimal!	1.4.12 Best Practice
125	1.3	H1 – H6	bzw. bei Handschriften größer!	Best Practice

Ausrichtung von Texten

ltr	rtl	Text-Art	Kommentar	Kriterium
links	rechts	Fließtext	Maximal sehr kurze Texte zentrieren!	Best Practice
links	rechts	H1 – H6	in Ausnahmen zentriert, kein Blocksatz!	Best Practice
zentriert		buttons	Button-Text, mobil über die ganze Breite!	Best Practice

rtl: in Sprachen die von **links nach rechts** gelesen werden.

ltr: in Sprachen die von **rechts nach links** gelesen werden.

Zentrierter Fließtext kann sparsam eingesetzt werden, maximal ca. 3–5 Zeilen lang!

Blocksatz hat im Webdesign NICHTS verloren!

Maximale Zeilenlänge

Gerade auf großen Bildschirmen maximal **75–85 Zeichen pro Zeile** (ggf. in Google Drive oder Word Zeichen **MIT Leerzeichen** zählen lassen).

Einheit im CSS: ch

Beispiel für die maximale Breite eines Containers für Fließtext: max-width: 85ch

Sinnvolle Abstände angeben

Die Einheiten % und rem bzw. em sind auch in den Innen- und Außenabständen von Elementen zulässig. Damit kann es jedoch zu Problemen kommen!

Beispiel für die Folgen schlecht gewählter Abstände

Das folgende Beispiel nutzt **Schriftgröße 1rem, Außen- und Innenabstand 1rem.**

Umgewandelt für dieses Programm ist die Schriftgröße 12pt, der Abstand 0,4233 cm.

Darstellung bei Schriftgröße 16px:

Lorem mi ipsum dolor sit amet, consectetur adipiscing elit. Aenean ligula commodo eget dolor. Aenean massa.
--

Im Beispiel passen bei Schriftgröße 16px ungefähr 75 Zeichen in eine Zeile.

Darstellung bei Schriftgröße 72px (4,5 mal größer als 16px):

Schriftgröße $12\text{pt} \cdot 4,5 = 54\text{pt}$, Abstände 1,905 cm

Lorem mi ipsum dolor sit..

Bei dieser Einstellung passen nur noch 7–10 Zeichen Text nebeneinander, auch in der Höhe sind die Abstände größer geworden, so dass vertikale Abstände ebenfalls größer werden.

Positiv-Beispiel

Das folgende Beispiel nutzt **Schriftgröße 1rem, Außen- und Innenabstand 16px.**

Umgewandelt für dieses Programm ist die Schriftgröße 12pt, der Abstand 0,4233 cm.

Bei dieser Einstellung ändert sich an unserer ursprünglichen Darstellung Nichts:

Darstellung bei Schriftgröße 16px:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean 1234566 commodo ligula eget dolor. Aenean massa.

Im Beispiel passen bei Schriftgröße 16px knapp 75 Zeichen in eine Zeile.

Darstellung bei Schriftgröße 72px (4,5 mal größer als 16px):

Schriftgröße $12\text{pt} \cdot 4,5 = 54\text{pt}$, Abstände 1,905 cm

Lorem mi ipsum dolor sit amet,

Bei dieser Einstellung passen nur noch 10–12 Zeichen Text nebeneinander.

Farbkontraste

Verhältnis	zu Schwarz	zu Weiß	Einsatzgebiet	Kriterium
21:1	#FFFFFF	#000000	Hochkontrast-Modus	maximum
7:1	#959595	#595959	Minimum für Text AAA	WCAG 1.4.6 AAA
4.5:1	#757575	#767676	Minimum für Text AA	WCAG 1.4.3 AA
3:1	#595959	#959595	Minimum für Elemente	WCAG 1.4.11 AA
0,5:1	#2D2D2D	#d1d1d1	Zeilen-Hinterlegungen	–

Hinweis: Laut WCAG-Kriterium [1.4.3](#) AA reicht für große Schrift ein Kontrastverhältnis von 1:3. Da die Schriftgröße jedoch nicht in der Hand der Designer:innen liegt, gilt 1:4,5!

Das “Favicon” Favoriten-Icon

Das Favoriten-Icon wird auf verschiedenen Geräten und Betriebssystemen unterschiedlich groß und an verschiedenen Orten dargestellt:

- im Tab (Reiter) der Browser in der kleinsten Version: 16*16 bzw. 32*32 Pixel
- im Lesezeichen-Menü der Browser
- Wenn ein Lesezeichen erstellt, oder eine Web-App heruntergeladen wurde: auf den Menü-Screens des Gerätes als Lesezeichen-Icon:
 - auf iOS mit abgerundeten Ecken
 - auf Android als ein Quadrat, oder als freigestellte Form.

Das Favoriten-Icon basiert zumeist auf dem Logo der Webseite.

Implementierung

Bei Webseiten müssen die Favoriten-Icon-Dateien in verschiedenen Größen und Formaten am besten auf oberster Ebene im gleichen Verzeichnis wie die Index-Seite liegen.

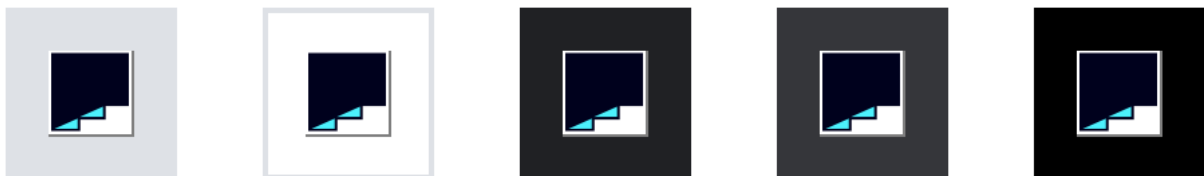
Damit alle Browser das jeweils dafür gedachte Icon finden, ist es notwendig, die Links dazu in den Metadaten im <head> anzugeben.

Aspekte der Barrierefreiheit

Es ist nicht möglich, unterschiedliche Icons für unterschiedliche Farb-Modi oder Hintergründe anzugeben.

Entsprechend muss ein und dasselbe Icon auf unterschiedlich hellen (im Tab/Reiter) und farbigen Hintergründen (auf dem Menü-Screen) funktionieren.

Beispiele



Überschneidungen mit Apps

Bei iOS-Apps ist es möglich, mehrere Icons zu hinterlegen und die Nutzenden selbst entscheiden zu lassen, welches Icon wünschen.

Das ginge ggf. auch für unterschiedliche Modi, wenn die Nutzenden dies wünschen.

Links und Materialien

- Sketch-Vorlage und Link auf Figma-Vorlage für unterschiedliche Hintergründe:
<https://github.com/absichtbar/favicon>
- SVG-Icons in notwendige Größen umwandeln und HTML-Code erzeugen:
<https://realfavicongenerator.net/>

Richtungen anzeigen und nutzen

Mitunter gilt es, einen Pfeil einzusetzen, um eine Richtung anzuzeigen, oder aber ein Logo oder eine Illustration enthält Elemente, die in eine bestimmte Richtung weisen.

Dabei gilt es darauf zu achten, dass diese Richtungsanzeigen logisch sind, oder unseren Lesegewohnheiten entsprechen! Entsprechend sind Pfeile bei Schriften die von rechts nach links gelesen werden anders zu behandeln, als bei Schriften die von links nach rechts gelesen werden!

Implementierung

Manche Pfeile und Elemente gehören für uns logisch vor oder nach dem Text platziert.

Aspekte der Barrierefreiheit

Wenn Zeichen und Beschriftung nicht übereinstimmen, wird es kompliziert. Speziell auf Seiten in Leichter Sprache ist es ein wichtiges Thema!

Einfach die Beschriftung wegzulassen, zählt aber in keinsten Weise!

Überschneidungen mit Apps

Betrifft auch z.B. klassische Baumstrukturen in App-Menüs.

Beispiele

- Logo der Deutschen Bank: Es geht aufwärts!
- [< Zurück] ist besser als [Zurück >]
- [Weiter >] ist besser als [< Weiter]
- Bild einer Person oder eines Tieres etc. eher von links nach rechts im Bild bewegen
- Link nach oben zum Seitenanfang: ^ hoch zeigen
- Blickrichtung von Personen bei Bildern auf den CTA (Call to action) lenken
- Pfeile zum Öffnen eines geschlossenen Klappmenüs:
 - > vor dem Text wenn es zu ist ODER
 - Erst der Text und dann am Ende der Pfeil zurück auf den Text zeigend! <
- Pfeile zum Schließen eines geöffneten Klappmenüs:
 - v eine Variante ist: der Pfeil zeigt auf den nun sichtbaren Inhalt
 - ^ oder der Pfeil zeigt dahin, wo der sichtbare Bereich zum schließen hin soll
- Karussells:
 - [>] Bild [<] ist üblich, zeigt aber nicht die Richtung in die gewipet werden muss
 - [>] Bild [<] würde die Richtung aus der das Bild kommt eigentlich korrekt anzeigen

Links und Materialien

Keine.

UI-Element Hero-Image (ohne Text)

Ein "Hero-Image" (Held:innen-Bild) ist ein Bild, das am Anfang einer Seite platziert wird und oft über die ganze Breite und teilweise die ganze Höhe des Browserfensters geht. Meist beginnt es direkt unter dem Header, manchmal wird es visuell mit dem Header kombiniert.

Das Hero-Image enthält aus Anbietendensicht ein "Key-Visual" mit dem Marken-Versprechen, oft auch das Logo, das positive Emotionen transportiert. Zum Beispiel:

- Spaß bei der Nutzung (Autos)
- Begehrte Produkte (Apple)
- Erfolg durch die Verwendung des Produktes (Sportschuh)

Dies ist nicht der Fall, wenn es ein generisches Bild ist, ohne inhaltliche Aussage. Wenn es nicht mal einen Hinweis auf den Seiteninhalt bietet, sollte lieber darauf verzichtet werden.

Generell sollte das Hero-Image nicht verhindern, dass vor dem Scrollen Informationen über den Seiten-Inhalt kommen...

Für das Responsive Webdesign gilt es auszuarbeiten, wie das Seitenformat sich zwischen den verschiedenen Screenbreiten und -höhen verhalten soll. Es gibt verschiedene Ansätze:

1. Festes Seitenverhältnis (100% Breite, Höhe abhängig vom Seitenverhältnis)
2. Variable Seitenverhältnisse (100% Breite, 100% Höhe)
3. Verschiedene Seitenverhältnisse (100% Breite, Höhe z.B. am Medium ausgerichtet)

Welche Version gewählt wird hängt von verschiedenen Faktoren ab (wenn die Vorgabe nicht auf Grund eines gekauften Themes bereits feststeht):

- Muss immer das ganze Motiv sichtbar sein?
- Gibt es Elemente im Motiv, die immer möglichst groß zu sehen sein sollen?
- Gibt es Elemente (siehe [UI-Element Hero-Image \(mit Text\)](#)) die das Motiv teilweise überdecken? Immer an der gleichen Position, oder unterschiedlich?

Implementierung

Als `<img>`

``

- Keine Größenangaben auf der HTML-Ebene angeben, die kommen aus dem CSS!
- Alt-Attribut sinnvoll füllen!

Vorteil: Alt-Text möglich

Nachteil: Es können keine unterschiedlichen Bildgrößen für verschiedene Bildschirmgrößen angegeben werden.

Kleine Größe groß darstellen: Qualitätsverlust durch Interpolation

Große Größe auf mobil darstellen: Ladezeit zu langsam, weil die Daten unnötig groß sind bzw. Nachhaltigkeit

Als <picture>

```
<picture>
<!-- referenziert x Bilder für unterschiedliche Situationen -->
<source srcset="pfad/bild.webp"
      type="image/webp"> <!-- unterschiedliche Dateiformate -->
<source srcset="pfad/bild_600.jpg"
      media="(min-width: 600px)"> <!-- bestimmte Größen -->
<source srcset="pfad/bild_dark.jpg"
      media="(prefers-color-scheme: dark)"> <!-- Farb-Modi -->
<source srcset="pfad/bild.jpg, pfad/bild_2x.jpg 2x" >
      <!-- für unterschiedliche Screen-Auflösungen -->
<!-- am Ende ein img als Default mit dem Alt-Text -->

</picture>
```

Als <div> mit Hintergrundbild

Bilder, die per CSS als Hintergrund von einem Element gesetzt werden können haben verschiedene Vor- und Nachteile:

- Vorteil:** Unterschiedliche Bilder für verschiedene Bildschirmgrößen etc. möglich.
Bei weniger Breite/Höhe können die Bilder beschnitten werden.
- Nachteil:** Kein Alt-Tag möglich (aber auch nicht immer nötig).
Nur nutzen, wenn das Bild WIRKLICH nicht beschrieben werden soll.

```
<div class="hero">Text mit CTA</div>
.hero {
  background-image: url(pfad/bild.png)
}
```

CSS-Angaben für responsive Hero-Images

Wenn die Höhe gefüllt werden soll, die Breite proportional sein:

```
img {
  width: auto;
  height: 100vw; // vw => viewport height
}
```

Wenn z.B. ein sticky header darüber steht:

```
img {
  width: 100%;
  height: calc(100vh - 80px); // verschiedene Werte kombinieren
}
```

Das Seitenverhältnis fixieren:

```
img {
  aspect-ratio: 4/3; // Es wird Breite/Höhe angegeben: 16/9...
}
```

Aspekte der Barrierefreiheit

Wenn das Hero-Image animiert ist, oder eine Videosequenz enthält, dann sollte es sich von alleine nicht länger als drei Sekunden bewegen, oder (leicht) pausierbar sein.

Überschneidungen mit Apps

In Apps wird ein solches Motiv meist nur als “Splash-Page” verwendet, das automatisch ausgeblendet wird, sobald die App vollständig geladen und aktiviert wurde.

Beispiele

Der Artikel mit dem das responsive Webdesign begann hat ein Hero-Image, bei dem sich das Motiv an verschiedene Bildschirmbreiten anpasst:

<https://alistapart.com/article/responsive-web-design/>

Links und Materialien

<picture> <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/picture>

UI-Element Hero-Image (mit Text)

Manche Hero-Images dienen als Hintergrund für Text:

- als Überschrift oder einen Slogan
- als Teaser Text mit **Call-to-action (CTA)** Link (nur für Start- und Übersichtsseiten geeignet)
- als Teaser Text ohne Link als Info zum Seiteninhalt (z.B. auf Unterseiten)

Der Text sollte auf jeden Fall NICHT ins Bild gerendert werden, auch nicht mit einem zusätzlichen Bild darübergelegt!

Das hat die folgenden Nachteile, die auch nicht von Optical Character Recognition (OCR) ausgeglichen werden können:

- nicht für Screenreader/Vorlesefunktionen und SEO verfügbar
- bei Verkleinerung des Bildes – z.B. für Smartphones unlesbar!
- passt sich nicht an die Schriftgrößeneinstellung an und ist damit ebenfalls für viele unlesbar
- Bei Hochkontrastmodus keine Änderungen um den Text lesbar zu machen
- Muss für jedes Motiv, jede Sprache im Grafikprogramm erzeugt bzw. geändert werden.

Stattdessen sollte der Text in HTML gesetzt und per CSS sinnvoll gestaltet auf dem Bild positioniert werden. Dafür muss ggf. das Backend/CMS speziell eingestellt werden.

Implementierung

Gestalterisch

Wie die Lesbarkeit des Textes sichergestellt werden kann, hängt unter anderem damit zusammen, ob das Bild statisch ist, oder ob es von der Redaktion verändert oder gar von Nutzer:innen erzeugt werden kann ("User-generated-Content").

Möglich sind zum Beispiel

- vollflächige, opaque Hinterlegungen mit einem guten [Kontrast](#) zur Schriftfarbe.
- halbtransparente Flächen (siehe Hinweis zur Barrierefreiheit beim Header)
- Farbverläufe innerhalb des gleichen Helligkeitsspektrums
- CSS-Filter, mit denen das Bild z.B. abgedunkelt werden
Pro-Tipp: Dem Bild eine Hintergrundfarbe geben, damit der Text lesbar bleibt, wenn das Bild nicht geladen wird.
- Eine Outline oder ein Schatten (behutsam gestalten)

Im CMS

- Felder für die Überschrift der Textbox, des Textes, des Link-Ziels und des Link-Textes
- Möglichkeit zur Positionierungsänderung, wenn es das Motiv verlangt (nicht pixelgenau, aber als 9-Teilung: oben links, mitte, rechts, mitte links, mitte rechts, unten links, mitte rechts).
- Möglichkeit ein neues Motiv hochzuladen. Ob die verschiedenen Größen und Varianten serverseitig erzeugt oder einzeln hochgeladen werden müssen, gilt es

abzustimmen. Andere Farbvarianten – sofern nicht rein per CSS erreicht – müssen zusätzlich hochladbar sein.

Im CSS

Sicherzustellen, dass der Text in allen Größen und mit verschiedenen Farbvarianten immer gut lesbar bleibt. Auch die verschiedenen, im CMS steuerbaren Varianten zur Positionierung müssen hier aufgegriffen werden!

Im HTML

Hinsichtlich der Einbindung des Bildes gelten die Hinweise zur Implementierung vom [UI-Element Hero-Image \(ohne Text\)](#).

Aspekte der Barrierefreiheit

- Der Kontrast der Schrift zum Hintergrund muss immer hoch genug bzw. sich an die Einstellungen der Nutzer:innen anpassen.
- Der Hintergrund hinter der Schrift sollte nicht transparent sein, oder nur dann, wenn Nutzende “Reduzierte Transparenzen” nicht aktiviert haben.
- Die Position des Textblocks sollte idealerweise linksbündig und eher oben sein, damit er von Menschen mit Screen-Lupe besser gefunden wird.
- Der CTA muss mit der Maus, Tastatur und dem Finger auswählbar sein und Informationen über das Link-Ziel enthalten.

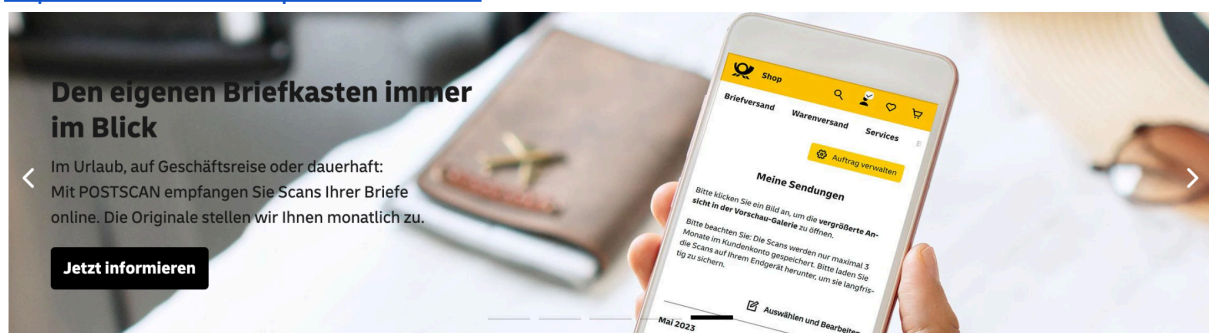
Überschneidungen mit Apps

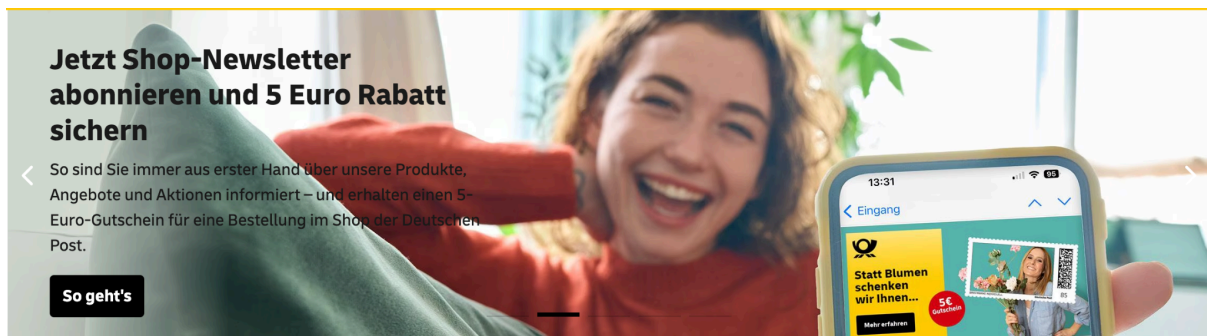
Apps haben auf Grund der anderen technischen Umsetzung und eingeschränkter Gerätebandbreite, weniger verschiedene Bildschirmgrößen und Ausrichtungen abzudecken. Die Hinweise zur Gestaltung und Barrierefreiheit sind auch hier relevant, weniger jedoch die Möglichkeit, Textblöcke unterschiedlich platzieren zu können.

Beispiele

Negative Beispiele für die Darstellung von Text auf redaktionellen Bildern

<https://www.deutschepost.de/de.html> Screenshots vom 25.04.2024





Das gleiche Motiv sieht im Hochkontrast-Modus "forced-colors" so aus:



Bzw. mit Dark-Mode so:



Die folgenden Prüfschritte zur Barrierefreiheit werden hier ignoriert:

- **Der Kontrast der Texte auf dem Hintergrund:**
[pruefschritt/bitv-20-web/bitv-20-web-9-1-4-3-kontraste-von-texten-ausreichend](https://www.w3.org/TR/WCAG21/#contrast-minimum)
<https://www.w3.org/TR/WCAG21/#contrast-minimum>
- **Bezüglich der Pfeile** (Die Aria-Label sind auf Englisch! aria-label="Previous slide"):
[pruefschritt/bitv-20-web/bitv-20-web-9-1-4-3-kontraste-von-texten-ausreichend](https://www.w3.org/WAI/WCAG21/Understanding/non-text-contrast.html)
<https://www.w3.org/WAI/WCAG21/Understanding/non-text-contrast.html>

Links und Materialien

Keine.

UI-Elemente Karussells

Slider mit mehreren Bildern, die durchgegangen werden können;

Es gibt mehrere inhaltliche Sub-Typen:

- A. Nur Bilder => Fotogalerie
- B. Bild mit Text => Informationsdarstellung
- C. Bild mit Text und CTA => Werbung bzw. Teaser

Zwischen den Bildern wird unterschiedlich gewechselt:

- A. Manuell => Bedienbarkeit muss sichergestellt sein
- B. Automatisch => Die Verständlichkeit und Bedienbarkeit können darunter leiden!

Ein Karussell kann:

- Platzierung und Ort des Hero-Images haben
- wie ein Hero-Image mit und ohne Text vorkommen
- im Inhaltsbereich vorkommen
- unterschiedlich hohe oder immer gleich hohe Elemente enthalten
- Slides über die ganze Breite, oder auch mehrere Kacheln nebeneinander darstellen

Ein Karussell sollte:

- Maximal 5–7 Slides enthalten. Ausnahme: Bildergalerien, z.B. AirBnB, Immoscout ...
- eine Möglichkeit enthalten zu erfassen, wie viele Slides es enthält, zum Beispiel durch Indikatoren in Form von Punkten, Miniaturansichten, horizontaler Scrollbar, Angabe das wievielte von wie vielen Slides gerade sichtbar ist
- auch über die Punkt-Indikatoren und Scrollbar bedienbar sein
- Pausierbar sein
- mit allen Arten der Bedienung steuerbar sein mit: Spracheingabe, Mausklick, Touch, Tastatur/Fokus
- nach Nutzer-Interaktion stehen bleiben
- optisch klar als eine Einheit erkennbar sein
- nicht der einzige Zugang zu den darin enthaltenen Inhalten sein!

Vorteile:

- Mehr Inhalt als nur 1 Bild auf kleinem Raum
- Es werden mehrere Angebote angezeigt ... WENN nicht weiter gescrollt wird

Nachteil:

- Wenn Nutzende das Karussell nicht komplett konsumieren, dann versteckte Dinge die eigentlich wichtig sind! => Dopplung auf der Seite in Betracht ziehen!
- kann Menschen überfordern (zu viele Infos, Wartezeit wenn automatisch weiter bis es wiederkommt, manchmal zu schnell um Bilder wahrzunehmen/text zu lesen, Screenreader müssen teilweise informiert werden, aber nicht die ganze Zeit, es ist nicht immer von allen steuerbar)
- Geht gerade weiter, wenn Nutzende sich zum Klicken entschieden haben
- Längere Ladezeiten bei hochqualitativen Bildern – die Performance leidet!

Implementierung

Karussells müssen mit Fingerspitzengefühl und sehr sorgfältig implementiert werden.

Aspekte der Barrierefreiheit

Beim Karussell gelten wie immer alle Prinzipien zur Barrierefreiheit. Jedoch ist dieses Element recht komplex und es können viele Fehler gemacht werden!

Wahrnehmbar

- Am besten kein Auto-Play – die Intervalle sind immer zu schnell oder zu langsam!
- Ein Wechsel muss für Screenreadernutzende nachvollziehbar sein, darf aber nicht die ganze Zeit angesagt werden, wenn mit dem Inhalt interagiert wird!

Bedienbar

- Das Karussell muss anhaltbar bzw. gezielt steuerbar sein – unabhängig von der Art der Bedienung!
- Die Pfeile müssen mit allen Bedienarten bedient werden können und für Screenreadernutzende in der korrekten Sprache mit Text ausgezeichnet werden, der einen Hinweis gibt, was ein Klicken auf den Pfeil bewirkt.

Verständlich

- Es muss deutlich sein, was zum Karussell gehört und wie es bedient werden kann
- Für Webseiten in Leichter Sprache sind Karussells grundsätzlich nicht geeignet!

Überschneidungen mit Apps

Karussells werden oft für das "Onboarding" verwendet.

Beispiele

Instagram enthält verschiedene Karussell-Gestaltungen:

- Werbung in Stories: hier ist der Link immer der gleiche, nur die Motive ändern sich.
- Beiträge können 10–20 Slides enthalten
- Karussell-Werbung in Vollbild, aber mit "Vorschau" auf alle anderen Produkte unten drunter (diese sollte Mobil eigentlich eine Mindest-Größe von 44 pixeln laut WCAG haben, laut Apple sogar 48px, sind bei bei Instagram aber kleiner...)

Links und Materialien

- Ein Skript für Karussells, für Screenreader gut umgesetzt! Dokumentation auf der Seite: <https://accessible360.github.io/accessible-slick/>
- Bei Eigenentwicklungen den folgenden Prüfschritt beachten!
bitvtest.de/pruefschritt/bitv-20-web/bitv-20-web-9-4-1-2-name-rolle-wert-verfuegbar

UI-Element Sticky Elements

Als Sticky Elemente werden Bereiche bezeichnet, die sich nicht von der Stelle rühren, wenn Nutzende scrollen.

Es gibt verschiedene Elemente, die als Sticky gesetzt werden können:

Sticky Header

Vorteile:

- Das Ziel von einem Sticky Header ist es, dass die Navigation immer erreichbar ist. Das ist mit der Bedienung mit Maus, Finger und Spracheingabe möglich. Für Screenreader ist es irrelevant, weil damit immer gut zur (korrekt ausgezeichneten) Navigation gesprungen werden kann. Für Menschen, die jedoch auf der Tastatur hauptsächlich mit dem Tabulator vor- und zurück navigieren, entfällt dieser Vorteil!

Nachteile:

- Manchmal ist der Bereich, der stehen bleibt zu groß – es bleibt kaum "Real Estate" (also Bildschirmplatz zum Scrollen) übrig.
- Beim Scrollen werde manchmal kleiner (runterscrollen), manchmal nur sichtbar, wenn hoch gescrollt wird. Dies führt oft dazu, dass genau das, was gerade zum Lesen positioniert wurde, überdeckt wird.

Sticky Footer

Kann ebenso wie ein Sticky Header zu viel von der verfügbaren Bildschirmhöhe überdecken.

Sticky Overlay/Floating Elements

Einige Webseiten enthalten einzelne oder mehrere Links oder Buttons, die immer an der gleichen Stelle positioniert sind und nicht mit scrollen.

Die Funktionen dahinter sind zum Beispiel:

- Cookie-Einstellungen ändern
- Chat starten
- Hilfe öffnen
- Barrierefreiheitsfunktionen ändern
- Social Media-Präsenzen aufrufen
- Auf Social Media teilen
- ...

Generell sollten nicht zu viele Elemente fix platziert werden. Wenn der Vorteil und die Wichtigkeit des Elements wirklich die Nachteile überwiegen, dann sollten die Elemente im Idealfall nicht in den Satzspiegel reinragen, erst recht nicht von beiden Seiten. Das macht den kompletten Seiteninhalt schlecht lesbar und unzugänglich für verschiedene Zielgruppen!

Implementierung

Die Höhe von Sticky Elementen (bei Floating Elementes auch die Breite) sollte immer mit Pixeln angegeben werden, nicht mit rem anhand der Schriftgröße ungebremst mitwachsen und auch nicht mit der Einheit vh proportional zur Screen-Höhe festgelegt werden.

Es ist generell sinnvoll, die Fixierung nur dann vorzunehmen, wenn eine bestimmte Browserhöhe nicht unterschritten wird.
Teilweise ist es sinnvoll, nur die Navigation zu fixieren, nicht die volle Höhe des Headers samt Logo und weiteren Elementen.

Der Header sollte optisch immer vom Inhaltsbereich unterscheidbar sein! Halb- oder ganz transparente Header verwirren oder irritieren bestimmte Zielgruppen.

Sticky Header und Anker-Links

Der Browser fixiert den Header am oberen Bildschirmrand. Wenn auf der Seite Anker-Links vorhanden sind, mit denen z.B. zu einer Überschrift gesprungen werden kann, dann verschwindet diese Überschrift hinter dem Header. Um sie voll zu sehen, muss erst wieder nach oben gescrollt werden. Das irritiert und nervt, weil es eine zusätzliche Interaktion bedeutet. Mit der CSS-Eigenschaft `scroll-margin` kann dies jedoch verhindert werden:

HTML: Link auf die Überschrift und Überschrift:

```
<header>
<nav>...</nav>
</header>
<main>
<ul>
<li><a href="#kontaktinformation"> Kontaktinformationen </a></li>
</ul>
...
<h2 id="kontaktinformation">Kontaktinformationen</h2>
...
</main>
```

CSS-Angabe:

```
header { height: 80px; // die Höhe nicht variabel machen!
          position: sticky;
          top: 0px } // top ist optional, Abstand selten sinnvoll
h2[id] { scroll-margin-top: 80px;}
```

position: sticky vs. position: fixed

Mit `position: sticky;` wird ein Element zunächst in dem Bereich eingeblendet, in dem es im HTML-DOM (Document Object Model) steht, zum Beispiel weiter unten auf dem Screen. Erst wenn hoch gescrollt wird, bleibt es am oberen Rand des Browserfensters "hängen".

Ein Bereich kann auch mit `position: fixed;` fixiert werden. Dann jedoch wird er immer dort positioniert sein, nicht nur wenn der Abstand zum oberen Rand kleiner wird als der angegebene Bereich.

Sticky Header, Formulare und virtuelle Tastaturen

Generell ist es ein Problem, wenn ein Sticky Header zu viel Platz, vor allem von der verfügbaren Höhe, einnimmt. Besonders problematisch wird dies auf Smartphones, Tablets

und anderen Geräten, wenn eine virtuelle Tastatur zum Ausfüllen der Formulare benötigt wird. Die virtuelle Tastatur wird zumeist am unteren Bildschirmrand eingeblendet und überdeckt bei kleineren Geräten schon im Hochformat ca. 50% des Bildschirms. Im Querformat kann es sogar mehr sein. Wenn dann zusätzlich der Header fixiert wurde, ist von dem Formular, das ausgefüllt werden soll, fast gar nichts übrig.

Dies kann mit CSS einfach verhindert werden! Es reicht, die Stickyness nur dann zu aktivieren, wenn auf der Seite kein Formular vorhanden ist.

Wenn jede Seite zum Beispiel ein Formular für den Newsletter hat, dann kann die Art Formular, auf die es zutreffen soll noch genauer spezifiziert werden:

```
body:not(:has(form)) header { position: sticky; }
```

Ein weiterer Ansatz kann sein, den Header nur dann sticky zu machen, wenn Elemente des Formulars nicht den Fokus haben. Vereinfacht kann es davon abhängig gemacht werden, ob das Formular den Fokus aktuell enthält, vom genauen Element unabhängig. Dies kann jedoch bei einigen Bedienarten dazu führen, dass der Header zwischen zwei Fokus-Zuständen eingeblendet wird.

```
body:not(:has(form:focus-within)) header { position: sticky; }
```

Sticky Header und Hero-Images

Siehe [CSS-Angaben für responsive Hero-Images](#).

Sticky Footer

Technisch gesehen wird der Footer häufig fixiert, statt wirklich sticky zu sein.

Einen Footer tatsächlich sticky zu setzen und nicht einfach nur zu fixieren ist etwas komplizierter. Lösungen dafür werden zum Beispiel hier beschrieben:

<https://css-tricks.com/couple-takes-sticky-footer/> oder hier:

https://developer.mozilla.org/en-US/docs/Web/CSS/Layout_cookbook/Sticky_footers

Aspekte der Barrierefreiheit

- Sticky Header werden auf Webseiten mit Leichter Sprache oft eingesetzt.
- Sie sind unwirksam in Kombination mit einer Fokus-Navigation:
Es muss doch wieder zum Seitenanfang zurück getappt werden.
- Wenn das überdecken, was gerade aufgerufen wurde verwirre
- Wenn sticky Elemente (generell) nicht gut vom Inhalt abgesetzt sind, erschweren sie die Orientierung auf der Seite und die Lesbarkeit des Inhaltes.

Überschneidungen mit Apps

Auch bei Apps werden Bereiche des Screens fixiert. Speziell der Header und im Footer die Task-Bar. Da Apps jedoch insgesamt auf weniger Scrollen ausgelegt sind, gilt es hier vor allem die Position von kleinen sticky Overlays zu beachten!

Beispiele

Die Dokumentation von Mozilla zum Thema Scroll-margin-top nutzt einen fixierten Header ohne Scroll-margin-top selbst anzuwenden – Stand April 2024 wird die Überschrift “Syntax” erst sichtbar, wenn wieder etwas hoch gescrollt wird – Update Mai 2025 wurde das Problem gefixt:

<https://developer.mozilla.org/en-US/docs/Web/CSS/scroll-margin-top#syntax>

Links und Materialien

Keine.

Inhalts-Elemente von Webseiten

Äußere Strukturelemente

<Doctype! html>

- Informiert den Browser über die Kompatibilitätsstufe des HTMLs

<html>

- Umschließt den gesamten Webseiten-Code
- Enthält das Attribut lang (Sprache)

<head>

Die Informationen aus dem <head> sind nicht direkt auf der Seite sichtbar!

- Zeichenkodierung
- Viewport-Angabe: width=device-width steuert die Skalierung auf mobilen Geräten
Barrierefreiheit: Skalierung darf nicht eingeschränkt werden!
- <title> – Name der Unterseite - Name des Webauftritts (ohne Sonderzeichen)
- CSS-Links
- JavaScript
- Meta-Angaben:
Beschreibung und andere Infos für
 - SEO
 - Social Media (Auch Vorschaubilder mit Alt-Text!)
 - Favoriten-Icons etc.
- ~~Base-URL~~ nicht angeben, da sonst Anker-Links, z.B. von Sprungmarken nicht funktionieren

<body>

- Der sichtbare Bereich der Seite mit allen Inhalten und Bildern etc.
- JavaScript das nicht am Anfang benötigt wird

Struktur-Elemente – Landmarks

Alle Inhalte einer Webseite sollten Teil einer Landmark sein.

Hilfs-Inhalte, die außerhalb der Landmarks liegen, werden als <section> integriert!

Landmarks, die nicht einmalig sind erhalten ein spezielles Label!

<header> Headerbereich

Inhalte im Headerbereich

- Logo der Seite – ggf. mit Namen und Claim als Text
- Navigation – Hauptkategorien und Meta-Navigation
- ggf. Suche
- Sprachwechsler

Optisch sollte der Header-Bereich vom Rest abgegrenzt werden. Z.B.:

- Hintergrundfarbe
- Border unterhalb des Bereiches oder rings herum => mindestens im Forced-Colors-Modus bei fixierten Headern
- Kann fixiert werden – Scroll-margin-top beachten
- Wird manchmal beim Runterscrollen aus- und hoch wieder eingeblendet

<nav> Navigation

- oft im Header integriert
- Alle Navigationslinks sind zusätzlich in einer Liste strukturiert – diese kann auch Unter-Listen enthalten, die die Ebenen widerspiegeln!
- Maximale Hauptkategorien:
 - sieben bei Standard-Sprache
 - fünf bei Leichter Sprache
- Muss in allen Ebenen bedienbar sein mit:
 - Maus
 - Tastatur
 - Touch
- Alle Unterpunkte, die aufklappen, müssen auch schließbar sein!
- WICHTIG: es muss ausgezeichnet werden, wo ich mich in dem Baum befinde!
- Kann einen Link auf die Suche ODER den Such-Slot selbst enthalten!
- Mobil oft eingeklappt – ausklappen per CSS oder JavaScript, besser nicht duplizieren!

Bold, unterstrichen, mit Farbe hinterlegt, visuell UND im Code muss aria-current="page" angegeben werden!

Tipp: die visuelle Auszeichnung mit CSS an das [aria-current=page] knüpfen!

<main> Hauptbereich (einmalig)

Mögliche Inhalte:

- Texte
 - Überschriften
 - Absätze/Paragraphen
 - Listen (unordered und ordered)
 - Definitions-Listen z.B. Glossar-Einträge
 - Zitate
 - Klappmenüs "Accordeon"
 - Vorformatierter Text, z.B. Code
 - Adressen
 - Zeitangaben
 - ...
- Bilder und Icons (Kategorien oder Hilfestellungen) z.B. PNG oder JPG, SVG...
- Produkt-Karten/Kacheln kombinieren Text, Bild und Links
- Hyperlinks auf
 - Interne (Unter-) Seiten
 - Anker-Links (beginnend mit #) von Sprungmarken und Inhaltsverzeichnissen
 - Externe Webseiten, z.B. Paypal-Bezahllinks
 - Share-Links – enthalten den Link auf diese Seite
 - Dokumente
 - PDF
 - Excel
 - ...
- Video
- Audio z.B. Podcasts
- Formulare
 - s.u.
- ...

Vorteil davon, es im Hauptbereich zu haben ist die gute Auffindbarkeit für:

- SEO
- Screenreader

<footer> Fußzeile

Mögliche Inhalte – zum Teil abhängig vom Zweck der Seite!

- Rechtliche Links
 - Impressum (außer es ist rein privat, Ladefähige Adresse ist Pflicht!)
 - Datenschutzinformation
 - AGB – wenn vorhanden
 - Barrierefreiheitserklärung
- Sitemap (Links der Hauptnavigation)
- Newsletteranmeldung
- Profile im Internet z.B. auch Buchungsplattformen
- Adressdaten
- Öffnungszeiten
- ...

<aside> Zusatzinformationen “Marginalspalte”

“Nice to know” Informationen mit Bezug zu den Hauptinformationen, aber weniger Wichtigkeit für z.B. SEO:x

- Kategorien vom Blog
- Öffnungszeiten
- Termine von Veranstaltungen
- ...

Mobil: Über oder unter <main> darstellen?

- Abhängig vom Inhalt
- Darüber als Klappmenü darstellen

<section> Unterteilung – nur mit Aria-Label eine Landmark!

Kann direkt im <body> oder als Unterpunkt der anderen Landmarks eingesetzt werden.

Direkt im Body z.B. für die **Sprungmarken**.

Im <main> z.B. für die Sprungmarke zum Seitenanfang!

Manche Seiten unterteilen alles in Sections, was eine eigene Überschrift hat. Dann sollte die Section auch ein Label haben!

<article> Keine Landmark, aber fast genauso wichtig!

Unterscheidung²⁸ zur <section>.

<section> und <article> sollten immer mit Überschrift <h2> oder <h3> beginnen.

Dies kann visuell ausgeblendet werden!

²⁸ <https://www.smashingmagazine.com/2022/07/article-section-elements-accessibility/>

Block-Elemente für einfache Inhalte

<div> Block-Element ohne strukturelle Bedeutungsebene

<H1> – <H6> Überschriften

<p> Paragraphen/Textabschnitte

Listen

 Ungeordnete Listen

 geordnete Listen

<dl> Definitions-Listen

<details> Klappmenüs

<pre> Vorstrukturierter Text

Block-Elemente für komplexe Inhalte mit Unter-Tags

<form> Formulare

<fieldset>

<input>

<textarea>

<button>

<table> Tabellen

Nur in Ausnahmefällen einsetzen und auch dann nur für Daten-Tabellen!

<th>

<tr>

<td>

Inline-Elemente

 Inline-Element ohne strukturelle Bedeutungsebene

<a> Links

<mark> Text markieren

<code> Text wird mit Mono-Space-Schrift dargestellt

 (kursiv) betonter Text

 (bold) hervorgehobener Text

Anhang

Glossar

Button vs. Link Funktionalität und Darstellung

Ein Link ruft einen anderen Inhalt auf:

- eine andere Seite
- eine bestimmte Stelle auf einer Seite
- ein Dokument, das im Browser angezeigt werden kann, wird angezeigt – z.B. PDF oder Bilddateien wie JPG, PNG, WebP
- ein Dokument, das nicht im Browser angezeigt werden kann, wird heruntergeladen – z.B. DOC, DOCX, XLS, XLSX etc.

Ein Button

- ändert eine Einstellung
- schickt ein Formular ab

Viewport

Als Viewport wird der sichtbare Bereich einer Webseite bezeichnet. Also das, was innerhalb einer bestimmten Bildschirmgröße und als Fläche im Browser abzüglich der Ränder und Bedienelemente zur Verfügung steht.

Der Viewport wird in Pixeln gemessen, die nicht deckungsgleich mit den Bildschirmpixeln sind. Vielmehr ergibt sich die Viewportgröße aus den Bildschirmpixeln geteilt durch den DPR, der Device-Pixel-Ratio. Außerdem besteht in einigen Browsern (auf dem Mac Chrome, Opera und Edge)²⁹ eine Abhängigkeit vom Zoomfaktor.

Formel

$$\frac{\text{Bildschirmpixel}}{\text{Device-Pixel-Ratio}} * \text{Zoomfaktor} = \text{Viewport}$$

Testmöglichkeit

<https://whatismyviewport.com/> zeigt den Viewport des aktuell genutzten Browserfensters.

Korrekte Umsetzung im HTML

im <head> einer jeden Webseite muss die folgende Zeile stehen:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Nur dadurch ist sichergestellt dass eine Webseite auch mobil in 100% dargestellt wird.

²⁹ Die Browser Safari und Firefox beziehen den Zoomfaktor nicht in die Berechnung des Viewports ein!

Hilfreiche Online-Tools und Quellen für digitales Design

Diese Liste ist nicht erschöpfend, es gibt noch viel mehr wichtige Quellen!

Barrierefreiheits-Kriterien

- **WCAG – Englische Kurzreferenz**

Die Zahlen die in diesem Dokument verlinkt sind beziehen sich auf:

<https://www.w3.org/WAI/WCAG22/quickref/>

- **Prüfschritte des BIK-BITV**

Die Zahlen aus der WCAG haben in der BIK-BITV ein 9. vorangestellt.

Die Prüfschritte, die mit 5, 6, 7, 11 oder 12 beginnen gibt es nur in der BIK BITV bzw. auch in der weitgehend übereinstimmenden Tabelle A1 der EN 301 549.

<https://bitvtest.de/pruefverfahren/bitv-20-web>

Design-Tools

- **Online-Collaboration-Design-Tool Figma** <https://www.figma.com/>

Unbedingt nutzen damit das Tool seine ganzes Potential zeigen kann:

- Formatvorlagen für Texte und Stilvorlagen für Elemente
- Auto-Layout um Anordnungen und Abstände zu kontrollieren
- Komponenten für alle Module erzeugen die öfter verwendet werden (Nicht nutzen um verschiedene Versionen gegeneinander zu testen!)
- Die verschiedenen Screengrößen nebeneinander stellen und bearbeiten
- immer in 100% arbeiten um ein Gefühl für die Proportionen zu bekommen!

Schriftart und Typographie

Schriftart

- Schrift mit Fokus auf Sehschwächen: **Atkinson Hyperlegible New + Mono**
<https://brailleinstitute.org/freefont>
- Schrift mit Fokus auf Lese-Rechtschreib-Schwäche: **Open Dyslexic**
<https://opendyslexic.org/>
- **Luciole Vision**
<https://luciole-vision.com/>
-
- **Freefonts von Google Fonts:** <https://fonts.google.com/>
Hinweis: die Fonts müssen auf den eigenen Server kopiert werden, um den Datenschutz-Vorgaben der DSGVO zu genügen! Sonst 100€ Strafe möglich!
- **Korrekte Zeichen finden:** <https://symbll.cc/de/>
Hinweis: die Suche ist nicht gut, manchmal hilft die Suche im Englischen weiter!
- **Empfehlungen für barrierearme Schriften**
<https://accessibility.psu.edu/fontfacehtml/>

Der Artikel über barrierearme Monospace-Schriften

boundlessreading.org/an-accessible-monospaced-font-for-low-vision-developers/

verweist auf:

- **Gintronic** (ab 100€)
<https://markfromberg.com/projects/gintronic>

- **intel one mono** (free)
<https://github.com/intel/intel-one-mono/releases/>

Bei allen Schriften muss immer die Gültigkeit der Lizenz geprüft werden!

Wie kann ich eine Schrift hinsichtlich der Barrierefreiheit bewerten?

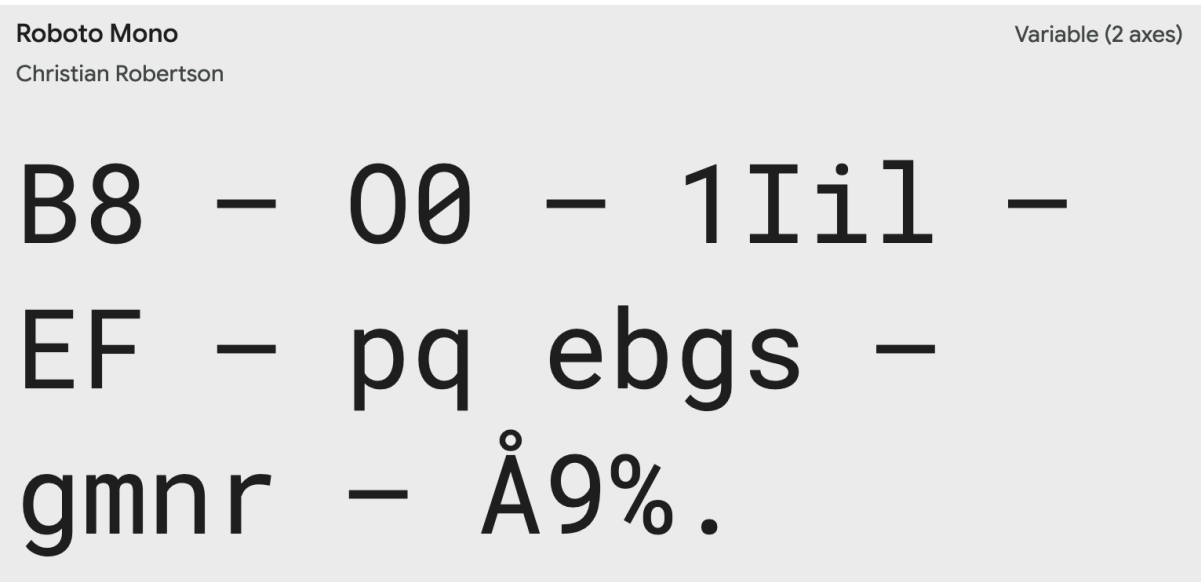
Der folgende String kann genutzt werden, um z.B. bei Google Fonts die verschiedenen Schriften zu beurteilen. Die Buchstaben in den unten stehenden Gruppen sollten jeweils gut voneinander unterscheidbar sein:

B8 – 00 – 1lil – EF – pq – ebgs – gmnr – Å9%.

Hier ein paar Beispiel-Screenshots:



Roboto: Großes l und kleines l nicht gut unterscheidbar.



Roboto Mono: Großes l und kleines l sind gut unterscheidbar.

Beide Roboto-Versionen: p und q sehr symmetrisch, die Ansätze an den Grundstrichen recht hoch. Die “Bäuche” von B und 8 unterscheiden sich kaum.

B8 – O0 – 1Iil – EF – pq
ebgs – gmnr – Å9%.

B8 – O0 – 1Iil – EF – pq
ebgs – gmnr – Å9%.

B8 – O0 – 1Iil –
EF – pq ebgs –
gmnr – Å9%.

Atkinson Hyperlegible hat klar unterscheidbare Glyphen, auch bei ähnlichen Formen.

B8 – O0 – 1Iil – EF – pq
ebgs – gmnr – Å9%.

Luciole erfüllt einige Ansätze von Atkinson Hyperlegible, jedoch sind ihr gegenüber ein paar Schwächen erkennbar. Die Unterschiede zwischen den folgenden Buchstaben fallen eher gering aus: E und F, p und q.

Weitere Quellen zu barrierefreier Typografie

<https://accessibility.psu.edu/fontfacehtml/>

<https://apastyle.apa.org/style-grammar-guidelines/paper-format/accessibility/typography>

Farben

Farbwirkungen

- **Umfangreiche Liste inklusive kultureller Bedeutungen**
<https://www.webdesign-journal.de/farbwirkung/>

Farbkontraste

- **Die verschiedenen Kontrastarten samt Beispielen**
<https://www.webdesign-journal.de/farbkontraste/>

Farben erkennen

- **Ishihara-Farbtafeln** für den Selbsttest (nur Rot/Grün, nicht eindeutig)
https://www.color-blindness.com/ishihara_cvd_test/ishihara_cvd_test.html
- **Farnsworth Munsell 100 Hue Test**
<https://www.colorliteness.com/farnsworth-munsell-100-color-blind-test-more>
- **Farben sortieren**
<https://www.color-blindness.com/color-arrangement-test>
- **Detail-Test Rot-Grün Farbsehschwäche**
<https://www.color-blindness.com/rgb-anomaloscope-color-blindness-test/>
- **Blau oder Grün?**
<https://ismy.blue/>

Statistiken zur Farbpräferenz

- **Welchen Farbmodus Nutzer:innen bevorzugen**
www.computerbase.de/news/betriebssysteme/dark-mode-nutzung-umfrage.83345/
- <https://kopf-hand.de/blog/webdesign/dark-mode-2024>

Input – was fehlt noch

- optische Mitte + vertikale Ausrichtung
- Medienbruch
- Breakpoint
- Transparenzen
- Animationen
- RIAN-Varianten
- Platzierungen von Menüs (Off-Canvas etc.)